

AI Product Evals Cheat Sheet

A practical PM reference for defining, running, diagnosing, and acting on AI evals.

Five rules to remember: define expected behavior before seeing outputs; severity matters more than one accuracy score; find the first wrong step before fixing; rerun failures plus neighboring and holdout cases; use evals to make product decisions, not just produce scores.

1. What an eval is

An AI eval tests whether probabilistic AI behavior is good enough for the product's intended use.

Normal software test	AI eval
Did the software do what it was programmed to do?	Did the AI behave correctly enough across realistic cases?

Keep deterministic guarantees and probabilistic AI behavior separate, even when both are part of the same end-to-end workflow.

2. What the PM owns

- What behavior matters and what counts as correct.
- Severity of different failures and which cases represent realistic risk.
- Quality bars for release and tradeoffs between competing metrics.
- Whether results are good enough for the intended use.

Engineering usually owns: repeatable execution, instrumentation, logging, and product integration.

AI can help: draft eval cases, variants, summaries, and initial analysis. Human product judgment approves expected behavior and decides what is acceptable.

3. Build the first eval set

Start small. Cover the behavior boundaries that matter most:

- Normal behavior
- Paraphrased behavior
- Edge cases
- Uncertainty
- Known failures
- Important negative cases where the AI should do nothing

Define expected behavior before looking at model outputs. Otherwise it is easy to unconsciously redefine success around whatever the model produced.

4. A simple scoring rubric

Dimension	Question
Supported	Is the conclusion actually supported by the evidence?
Complete	Did it catch the important information?

Dimension	Question
Uncertainty preserved	Did unknown, tentative, conditional, pending, or TBD information remain uncertain?
Review burden	Did it avoid unnecessary alerts, Reviews, or duplicate work?

Individual cases can be rated **Pass / Mixed / Fail**.

5. Severity beats one overall accuracy score

- **Low:** minor inconvenience.
- **Medium:** misleading or wasteful but easy to catch.
- **High:** could materially affect decisions or trust.
- **Critical:** could directly cause unsafe, irreversible, or externally consequential action.

A strong aggregate score can still fail the release bar if the remaining failure is severe enough.

State practice rule: unsupported consequential claims and meaningful loss of uncertainty are automatic failures. Other products may choose different zero-tolerance behaviors.

6. Grow evals around real failures

When a real failure appears, add nearby cases that test the same decision boundary rather than only the exact sentence that broke.

Boundary example	Expected distinction
Unknown	Remain unknown
Explicit 0	Allow 0 only when explicitly established
Tentative 10%	Do not treat as approved
Approved 10%	Propose the concrete value
Observed 8%, target unknown	Do not confuse actual performance with target
Prior target withdrawn	Replacement returns to unknown unless established

7. After a fix

- Rerun the original failure.
- Run nearby boundary cases.
- Run positive cases where the AI should confidently act.
- Run unseen holdout cases.
- Repeat high-risk or unstable cases several times.

This catches both **overcorrection** and fixes that only work on the examples used to design them.

8. Holdout cases and repeated runs

Holdouts: keep some cases that were not directly used to design the fix. Passing only tuned examples does not show that behavior generalizes.

Repeat more heavily: high-severity behaviors, previously failed cases, ambiguous boundaries, and cases with inconsistent output.

Routine lower-risk cases: one run may be enough in a normal cycle, with occasional repetition.

9. Trace failures before fixing them

The final bad output does not tell you which layer failed. Find the **first wrong step**:

Evidence -> context / retrieval -> model interpretation -> validation / post-processing -> classification -> product action

Example: if the model returns **null** and software converts it to **0**, changing the prompt will not fix the real problem.

Rule: do not change the prompt until you know the failure actually occurred at the model step.

10. What tracing should let you inspect

- Input evidence and context supplied to the model.
- Prompt / instruction version and provider / model version.
- Model interpretation.
- Validation or normalization.
- Review or action classification and final product action.
- Errors, timing, and a run / trace ID.

Ownership: PM defines what must be observable. Engineering usually implements the instrumentation.

11. Find silent misses

Visible bad outputs are easier to notice than cases where the AI should have acted but did nothing. Audit the negative cases.

- Sample no-action cases.
- Have humans judge whether doing nothing was correct.
- Track confirmed false negatives and severity.
- Turn meaningful misses into future eval cases.

Do not rely only on user reports or accidental discovery.

12. Useful metrics

Metric	What it tells you
Important-change miss rate	How often important things that should have been caught were missed.
Unsupported-claim rate	How often the AI invented or overstated something.
Uncertainty-loss rate	How often uncertain information became falsely certain.
Unnecessary-review rate	Of all generated Reviews, how many were clearly unnecessary.

Avoid treating one overall "accuracy" percentage as the whole story.

13. Set product-specific quality bars

For the State practice exercise, we chose:

Safety	Zero high-severity uncertainty failures
--------	---

Review burden	No more than 20% of Reviews should be clearly unnecessary
---------------	---

These are **not universal industry thresholds**. They were product decisions for this exercise.

14. Quality is multidimensional

A fix can improve one metric while hurting another. Example: uncertainty failures drop to zero, but unnecessary Reviews spike. That is not automatically a successful fix.

Check safety, correctness, usability, and human workload together.

15. Conservative escalation can be correct

Do not penalize the AI for failing to know information it was never given. If freshness or provenance cannot be reliably determined, surfacing a possible conflict for human review may be the safest reasonable behavior.

Evaluate behavior relative to what the system could reasonably know.

16. Practical PM eval loop